

Άσκηση 3: Δημιουργία του k29photo Πόρταλ για Διαχείριση Φωτογραφιών

Αθανάσιος Ζαφείρης (1115202400047)

Γεράσιμος Γαλανός (1115202400019)

Εισαγωγή

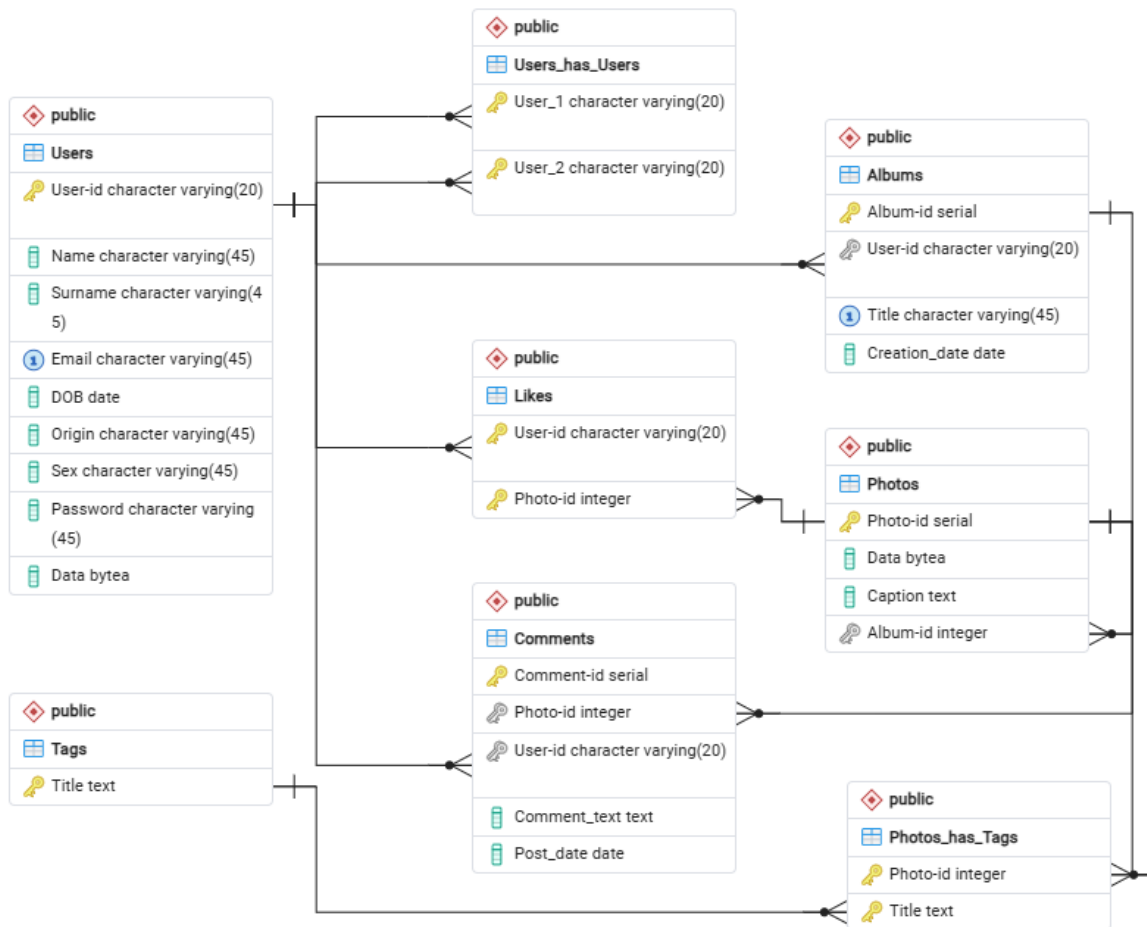
Ακολουθεί μια σύντομη σχετικά ανάλυση που αφορά στην 3η προγραμματιστική άσκηση του μαθήματος της Σχεδίασης και χρήσης βάσεων δεδομένων. Ξεκινάμε με την ανάλυση των πινάκων.

Πίνακες

Ακολουθεί εικόνα του ER διαγράμματος:

Προτού ξεκινήσουμε με την ανάλυση των πινάκων, η βάση σχεδιάστηκε ώστε να εξασφαλίζει την ακεραιότητα των δεδομένων χρησιμοποιώντας ξένα κλειδιά (Foreign Keys) με ON DELETE CASCADE, επιτρέποντας π.χ. τη διαγραφή ενός χρήστη να διαγράφει αυτόματα τα άλμπουμ, τις φωτογραφίες και τα σχόλιά του.

- **Χρήστες (Users):** Το μέρος όπου καταχωρούνται οι πληροφορίες των χρηστών της πλατφόρμας μας, που δίνονται κατά την εγγραφή τους στα σχετικά πεδία του `register.html`.
- **Φίλοι (User_has_Users):** Ένας πολύ απλός πίνακας ο οποίος αποθηκεύει τα μοναδικά IDs δύο χρηστών που είναι φίλοι μεταξύ τους. Δεν ισχύει όμως το εξής: Ο Α είναι φίλος με τον Β κι άρα ο φίλος είναι Β με τον Α. Με άλλα λόγια, θα έχουμε δύο εγγραφές αν ο Α έχει φίλο τον Β και το ίδιο ισχύει αντίστοιχα για τον Β με τον Α.
- **Albums:** Κάθε άλμπουμ είναι ξεχωριστό, με το δικό του ID. Ωστόσο, εξασφαλίζουμε ότι ο κάθε χρήστης μπορεί να έχει μοναδικό άλμπουμ με το ίδιο όνομα. Δηλαδή, αν υπάρχουν πολλαπλά άλμπουμ με το όνομα «Μαδρίτη 2025», θα ανήκουν σε ισάριθμους χρήστες.
- **Φωτογραφίες (Photos):** Τα δεδομένα τους αποθηκεύονται με την πραγματική δυαδική αναπαράστασή τους και καθεμιά φωτογραφία αποθηκεύεται σε ένα album (λόγω του `album-id`). Επίσης, φέρουν σειριακό ID για πιο απλή διαχείριση. Με την αποθήκευσή τους σε άλμπουμ, ανακατούμε και τους χρήστες που τις ανεβάζουν. Το πεδίο `caption`, αφορά στη λεζάντα που φέρει η κάθε φωτογραφία.
- **Ετικέτες (Tags):** Ο πιο απλός πίνακας της βάσης. Περιέχουν απλά τα ονόματα των ετικετών που είναι μοναδικά. Νέα εγγραφή καταχωρείται όταν κάποιος/α δημοσιεύει φωτογραφία με ετικέτα που δεν έχει εμφανιστεί στο παρελθόν.



Σχήμα 1: ER Διάγραμμα Βάσης k29photo

- **Φωτογραφίες με ετικέτες (Photos_has_Tags):** Μέσω αυτού του πίνακα βλέπουμε ποιες ετικέτες φέρουν οι φωτογραφίες. Πρωτεύον κλειδί του πίνακα αυτού είναι ο συνδυασμός του ID της φωτογραφίας με τον τίτλο της ετικέτας. Έτσι, εξασφαλίζουμε ότι μια ετικέτα μπορεί να χρησιμοποιηθεί για πολλαπλές φωτογραφίες αλλά μια φορά σε κάθε δημοσίευση.
- **Σχόλια (Comments):** Έχουν σειριακό ID καθώς και ID χρηστών και φωτογραφιών για να ξέρουμε ποιος αφήνει κάθε φορά σχόλιο. Επίσης, ένας χρήστης (συνδεδεμένος ή μη) μπορεί να αφήσει πολλαπλά σχόλια στην ίδια δημοσίευση ενώ ο κάτοχος της δημοσίευσης δεν μπορεί να τη σχολιάσει.
- **Likes:** Περιλαμβάνουν μόνο τα ID των χρηστών και των φωτογραφιών. Απαραίτητη προϋπόθεση για την υποβολή ενός like, είναι η πραγματοποίηση εισόδου με λογαριασμό στην πλατφόρμα.

Διασύνδεση

Βασιστήκαμε κατά κύριο λόγο στους οδηγούς που αναρτήθηκαν στην ιστοσελίδα του μαθήματος και στα όσα έδειξε η κυρία Παναγίδα στο Webex. Σε ένα αρχείο (`requirements.txt`) έχουμε σημειώσει το `flask` και την βιβλιοθήκη `psycopg2-binary`. Αυτά, μετά από την

ανάγνωση του `.txt` αρχείου, τα αξιοποιεί το `setup.sh` αρχείο για την κατασκευή της ιστοσελίδας.

Επίσης, στην βάση συνδεόμαστε με τις προκαθορισμένες τιμές για τους τοπικούς μας servers και δημιουργούμε ένα εικονικό περιβάλλον (`venv`) ώστε η εφαρμογή μας να τρέχει ανεξαρτήτως των ρυθμίσεων του κεντρικού συστήματος. Επιπλέον, ελέγχουμε για την ύπαρξη της βάσης κι αν δεν υπάρχει τότε δημιουργείται, ώστε να μπορούν να απεικονιστούν απευθείας στον browser του χρήστη.

Κώδικας εφαρμογής

Για την υλοποίηση της λογικής της εφαρμογής χρησιμοποιήσαμε ως επί το πλείστο την Python με κάποιες βιβλιοθήκες (`Flask`, `psycopg2`, `base64`, `datetime`). Δίνουμε φυσικά τα στοιχεία σύνδεσης με τη βάση και ξεκινάμε.

Σε γενικές γραμμές, η προσέγγιση είναι όμοια. Για κάθε σχεδόν δυνατότητα της εφαρμογής, αναπτύσσουμε ένα μπλοκ με μεθόδους (`GET` ή `POST` ή και τα δύο) καθώς αυτές καθορίζουν αν θα αντλήσουμε στοιχεία από τη βάση ή αν θα καταχωρήσουμε σε αυτήν. Αυτό εξαρτάται φυσικά από τη φύση της κάθε λειτουργίας. Χρησιμοποιούμε εντολές όπως:

- `session.get`: για άντληση π.χ. του ονόματος χρήστη κάποιου/ας που είναι ήδη συνδεδεμένος/η.
- `request.args.get`: για να χρησιμοποιηθούν τα διάφορα ορίσματα στα διάφορα ερωτήματα προς τη βάση.
- `request.form.get`: για να εισαχθούν δεδομένα στη βάση.
- `cur.execute`: για την εκτέλεση των ερωτημάτων.
- `get_db_connection` και `conn_cursor`: για τη σύνδεση με τη βάση και τους κέρσορες σε κάθε λειτουργία της εφαρμογής.
- `render_template`: η Python ανακτά τα δεδομένα από τη βάση και τα περνάει στα HTML αρχεία που είναι κάθε φορά γραμμένα. Είναι ο τρόπος που μπορούμε να προβάλλουμε αποτελέσματα για διαφορετικά ερωτήματα στην ίδια σελίδα (π.χ. στην `home.html`).
- `append`: τα αποτελέσματα των αναζητήσεων εντάσσονται σε dictionaries ώστε να είναι αναγνώσιμα από την HTML.
- `commit`: με αυτό εισάγουμε δεδομένα στη βάση εφόσον το `try` είναι επιτυχές.
- `rollback`: με αυτό ακυρώνουμε την εισαγωγή δεδομένων στη βάση αν το `try` είναι αποτυχές.
- `Base64`: μετατροπή των φωτογραφιών, που είναι αποθηκευμένες σε δυαδικά αρχεία (`BYTEA`), σε μορφή κειμένου ώστε να μπορούν να απεικονιστούν απευθείας στον browser του χρήστη.

Ερωτήματα

Για την υλοποίηση μιας εφαρμογής σαν αυτήν, είναι απαραίτητη η ενσωμάτωση μερικών ερωτημάτων στην εφαρμογή μας. Κάποια από αυτά είναι σύνθετα και πολύπλοκα κι αξίζει να τα αναλύσουμε:

Κορυφαίοι χρήστες: Βρίσκουμε τον αριθμό των δημοσιεύσεων των χρηστών με το ερώτημα:

```
SELECT count(*)
FROM "Photos" p
JOIN "Albums" a on a."Album-id" = p."Album-id"
WHERE u."User-id" = a."User-id"
```

και στη συνέχεια τον αριθμό των σχολίων σε φωτογραφίες άλλων χρηστών (εξάλλου δε σχολιάζουν δικές τους φωτογραφίες οι χρήστες) με το ερώτημα:

```
SELECT count(*)
FROM "Comments" c
JOIN "Photos" p ON p."Photo-id" = c."Photo-id"
JOIN "Albums" a ON a."Album-id" = p."Album-id"
WHERE c."User-id" = u."User-id" AND a."User-id" != u."User-id"
```

Αυτά τα δύο νούμερα τα αθροίζουμε και ονομάζουμε το άθροισμα **Total_Contribution** έτσι ώστε εμφανίσουμε το άθροισμα (με φθίνουσα σειρά) και τα ονόματα των κορυφαίων χρηστών της ιστοσελίδας.

Κορυφαίες ετικέτες: Με την ίδια λογική αλλά με πολύ πιο απλή υλοποίηση (δε χρειαζόταν καν η ένωση πολλών πινάκων) φέρνουμε τις κορυφαίες ετικέτες. Το **GROUP BY TP."Title"** αρκεί.

Σύσταση φίλων:

```
SELECT p2."User_2", u."Name", u."Surname", u."Data", COUNT(*) AS
    mutual_count
FROM "Users_has_Users" p1
JOIN "Users_has_Users" p2 ON p1."User_2" = p2."User_1"
JOIN "Users" u ON p2."User_2" = u."User-id"
WHERE p1."User_1" = %s
AND p2."User_2" != %s
AND p2."User_2" NOT IN (
    SELECT "User_2"
    FROM "Users_has_Users"
    WHERE "User_1" = %s)
GROUP BY p2."User_2", u."Name", u."Surname", u."Data"
ORDER BY mutual_count DESC;
```

Εδώ εφαρμόζουμε ένα Self-Join (ένωση ενός πίνακα με τον εαυτό του). Ενώνουμε τον πίνακα φιλίας δύο φορές (p1 και p2). Αν ο τρέχων χρήστης είναι ο A, το p1 βρίσκει τους άμεσους φίλους του (B). Το p2 βρίσκει τους φίλους (C) αυτών των φίλων (B). Στο **WHERE** γίνονται τα εξής:

1. Ο **p1."User_1" = %s** είναι ο χρήστης που επισκέπτεται τις συστάσεις φίλων.
2. **p2."User_2" != %s** αποκλείουμε τον εαυτό μας από τους φίλους που θα προταθούν.

3. Στο NOT IN αποκλείουμε τα άτομα που είναι ήδη φίλοι με τον χρήστη που επισκέπτεται τις συστάσεις φίλων.

Σύσταση Περιεχομένου ('You May Also Like'): Χρησιμοποιούμε το WITH που απομονώνει τη λογική σε δύο διαδοχικά στάδια εντός της βάσης για να βελτιστοποιήσουμε την ταχύτητα εκτέλεσης του ερωτήματος. Σε πρώτη φάση βρίσκουμε τις πέντε κορυφαίες ετικέτες του χρήστη που πλοηγείται στην ιστοσελίδα:

```
WITH TopUserTags AS (  
    SELECT PT."Title"  
    FROM "Photos_has_Tags" PT  
    JOIN "Photos" P ON PT."Photo-id" = P."Photo-id"  
    JOIN "Albums" A ON P."Album-id" = A."Album-id"  
    WHERE A."User-id" = %s  
    GROUP BY PT."Title"  
    ORDER BY COUNT(*) DESC  
    LIMIT 5  
)
```

Και στη συνέχεια αναζητούμε τις φωτογραφίες των υπόλοιπων χρηστών που περιέχουν έστω μια από τις πέντε ετικέτες που βρήκαμε παραπάνω:

```
SELECT P."Photo-id", P."Data", P."Caption", COUNT(PT."Title") AS  
    matched_tags,  
    (SELECT COUNT(*) FROM "Photos_has_Tags" WHERE "Photo-id" = P."  
        Photo-id") AS total_tags  
FROM "Photos" P  
JOIN "Albums" A ON P."Album-id" = A."Album-id"  
JOIN "Photos_has_Tags" PT ON P."Photo-id" = PT."Photo-id"  
WHERE A."User-id" != %s AND PT."Title" IN (SELECT "Title" FROM  
    TopUserTags)  
ORDER BY matched_tags DESC, total_tags ASC
```

Στο τέλος, με το ORDER BY εξασφαλίζουμε το διπλό κριτήριο ταξινόμησης που ορίζει η εκφώνηση.

Για άλλες περιπτώσεις: Όπου ζητούνταν διάφορες δικλείδες (π.χ. να μην μπορεί να σχολιάσει κάποιος/α δική του/της δημοσίευση) δουλέψαμε κατά κύριο λόγο με την Python. Για το παραπάνω παράδειγμα γράψαμε τις παρακάτω δύο γραμμές:

```
if owner_row and current_user and str(owner_row[0]) == str(  
    current_user):  
    return "Σφάλμα: Δεν επιτρέπεται να σχολιάσεις τη δική σου φωτογραφία!"
```

Αντίστοιχα, με τέτοια blocks έχουμε λειτουργήσει και για άλλες τέτοιες περιπτώσεις σαν την παραπάνω.

Επισημάνσεις – Παραδοχές

- Έχουμε βάλει τη συμπλήρωση ετικετών ως υποχρεωτικό πεδίο.
- Για πολλαπλές ετικέτες, τις γράφουμε χωρίζοντάς τες με κόμμα. Π.χ. nafplion, vacation.

- Με τα σειριακά IDs η βάση δημιουργεί εύκολα τα αναγνωριστικά των φωτογραφιών, των άλμπουμ και των σχολίων χωρίς να χρειάζεται περαιτέρω δική μας παρέμβαση.